6.1 Introduction

Document Purpose:
This document presents the architecture and detailed design for the "Pathfinder" project, a web application built to assist students, recent graduates, and experienced professionals in career exploration. Pathfinder provides tailored career guidance, job recommendations, resume analysis, and an AI-powered chatbot for real-time support. The application integrates OpenAI's API for natural language processing, Firebase for data management and authentication, and AWS for hosting and cloud services.

6.1.1 System Objectives

Objective Overview:
The primary goal of Pathfinder is to offer users a comprehensive platform for career planning and professional development. Key objectives include:

- Personalized career guidance based on user inputs and preferences.
- Real-time job recommendations and updates.
- Interactive resume feedback and improvement suggestions.
- An AI chatbot that answers career-related questions.
- Secure handling of user data and a reliable authentication system.

6.1.2 Hardware, Software, and Human Interfaces

1. **Hardware Interfaces:**
    - **User Devices**: Accessible on desktops, laptops, and smartphones (Android and iOS).
    - **Cloud Infrastructure**: AWS servers for backend processing, application deployment, and load balancing.
2. **Software Interfaces:**
    - **Frontend**: React.js for a responsive and interactive user interface.
    - **Backend**: Node.js with Firebase for data storage and authentication.
    - **AI Integration**: OpenAI's API for the chatbot feature.
    - **APIs**: Job recommendation APIs for accessing external job listings.
3. **Human Interfaces:**
    - **User Interface**: A GUI accessed via web browsers (Chrome, Firefox, Safari).
    - **Dashboard Layout**: Intuitive and personalized user experience on a React-based interface with modules for job search, resume analysis, and chatbot interaction.

6.2 Architectural Design

Pathfinder's architecture is structured using a Model-View-Controller (MVC) design pattern. This organization helps maintain clear separation between data handling, user interaction, and the business logic.

6.2.1 Major Software Components

1. **User Authentication Module**: Manages secure sign-in and registration using Firebase Authentication.
2. **Dashboard Module**: Hosts the primary user interface, allowing access to core features like career planning, resume analysis, and job matching.
3. **AI Chatbot Module**: Processes natural language inputs and generates responses with OpenAI's API.
4. **Job Recommendation Module**: Connects to external APIs for job listings relevant to user preferences.
5. **Data Management Module**: Uses Firebase to store user data securely, with AWS providing backend resources.

6.2.2 Major Software Interactions

- **Frontend-Backend Communication**: Uses HTTP requests to pass data between the React frontend and the Node.js backend.
- **Backend-Firebase Interactions**: Firebase manages user data and authentication, ensuring secure storage and real-time data updates.
- **Backend-OpenAI Integration**: Processes chatbot queries through OpenAI's API, enabling AI-driven responses.
- **Backend-Job API Integration**: Retrieves job listings relevant to the user's profile and preferences.

6.2.3 Architectural Design Diagrams

- **Use Case Diagram**: Highlights interactions for core features (authentication, dashboard, job matching).
- **Class Diagram**: Shows primary classes like `User`, `JobSearch`, `ResumeAnalyzer`, and `Chatbot`.
- **Deployment Diagram**: Illustrates component hosting on AWS, including database and API interactions.

6.3 CSC and CSU Descriptions

6.3.1 Class Descriptions

Introduction:
The following sections describe the classes used in the Pathfinder application, grouped by functionality and ordered by complexity.

1. **User**
    - **Fields**: userId, email, name, preferences
    - **Methods**: login(), register(), updatePreferences()
2. **JobSearch**
    - **Fields**: jobId, title, company, location, matchScore
    - **Methods**: fetchJobs(), filterByPreferences()
3. **ResumeAnalyzer**
    - **Fields**: resumeContent, feedback
    - **Methods**: analyzeResume(), generateSuggestions()
4. **ChatbotService**
    - **Fields**: query, response
    - **Methods**: sendQueryToAI(), formatResponse()

6.3.2 Detailed Interface Descriptions

- **User to Dashboard**: Facilitates user login, registration, and session management using Firebase Authentication.
- **Chatbot API Interface**: Connects the user's query to OpenAI, returning responses to the frontend.
- **Job Search Interface**: Interfaces with external job recommendation APIs to fetch listings based on user-specified criteria.

6.3.3 Detailed Data Structure Descriptions

1. **User Data Structure**
    - Contains user-specific data like userId, email, and preferences.
2. **Job Listing Data Structure**
    - Holds information on job opportunities, such as jobId, title, and company.
3. **Resume Feedback Data Structure**
    - Stores detailed feedback and suggestions for each resume section.

6.3.4 Detailed Design Diagrams

- **Class Diagram**: Comprehensive structure of classes with relationships and attributes.
- **Sequence Diagram**: Demonstrates how a user's query flows from frontend through backend to OpenAI and back.
- **Activity Diagram**: Outlines the process flow for job recommendation retrieval and display.

6.4 Database Design and Description

6.4.1 Database Design ER Diagram

An Entity-Relationship diagram representing the main entities like User, JobListing, and ResumeFeedback, demonstrating the relationships and data dependencies.

6.4.2 Database Access

Firebase serves as the primary database, accessed by the backend through secure API calls. Firebase Authentication and Firestore handle user records, and AWS supports any additional data storage and processing needs.

6.4.3 Database Security

Firebase Authentication ensures secure access, while Firebase and AWS implement encryption to protect sensitive data. All API calls to OpenAI and job listing services are secured with HTTPS.